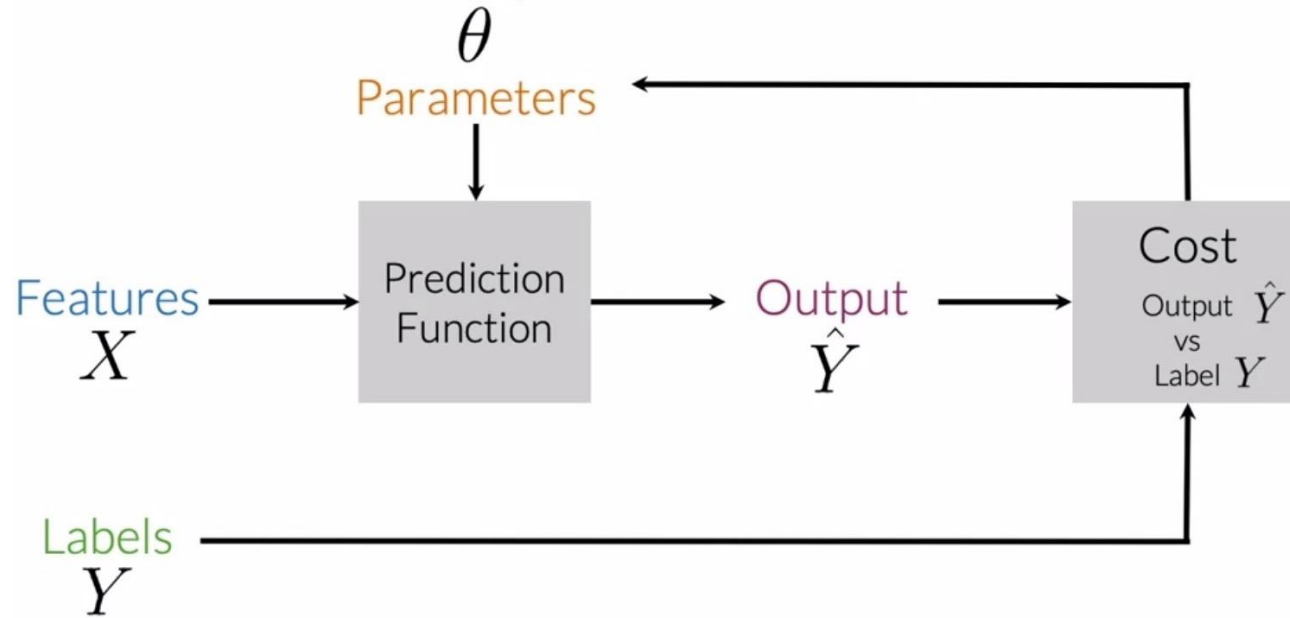


NLP

Week#07(1)

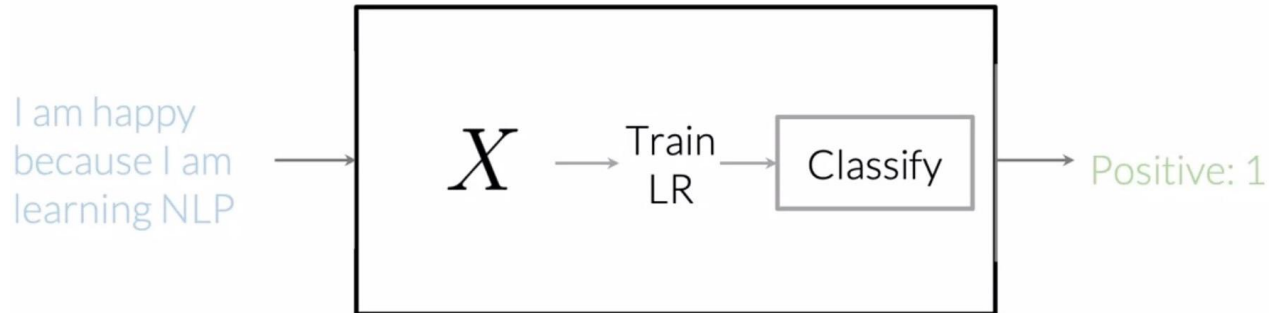
Supervised Machine Learning



Sentiment Analysis - Logistics Regression

Building a logisitic regression classifier that performs sentiment analysis on tweets can be done in 3 steps:

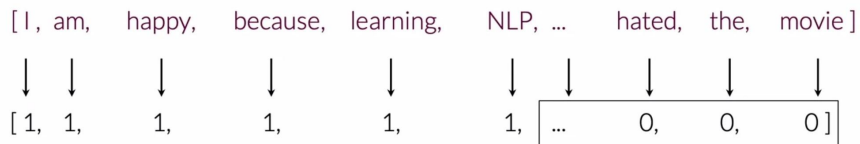
1. Extract features: process the raw tweets in the training set and extract useful features.
 - a. Tweets with a positive sentiment have a label of 1, while those with a negative sentiment have a label of 0.
2. Train: Train your logistic regression classifier while minimizing the cost.
3. Predict: Come up with predictions using your learned model.



1. Feature Extraction

1. Sparse Representations

I am happy because I am learning NLP



I am happy because I am learning NLP

$[1, 1, 1, 1, 1, 1, \dots, 0, \dots, 0, 0, 0]$
1 $\leftarrow |V|$

All zeros!

$[\theta_0, \theta_1, \theta_2, \dots, \theta_n]$
 $n = |V|$

- 1. Large training time
- 2. Large prediction time

2. Feature Extraction with Frequencies

I am sad, I am not learning NLP

$$X_m = [1, \sum_w \text{freqs}(w, 1), \sum_w \text{freqs}(w, 0)]$$

$\downarrow$

$$X_m = [1, 8, 11]$$

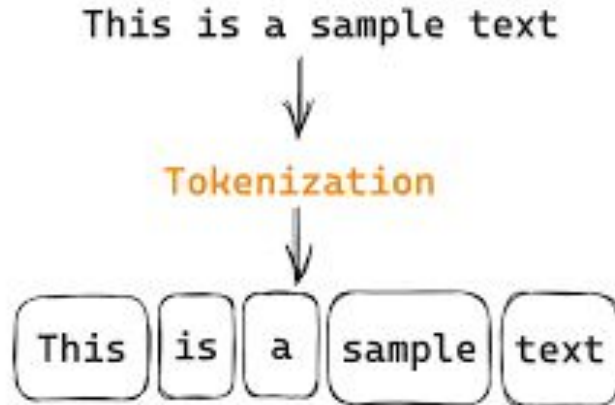
2. Preprocessing

There are four major preprocessing concepts in NLP:

- Tokenization
- Removing stop words and punctuation
- Stemming
- Lower-casing

Preprocessing - Tokenization

To tokenize a string means to split the strings into individual words without blanks or tabs.



Stop Words and Punctuation Marks

@YMourri and @AndrewYNg are
tuning a GREAT AI model at
<https://deeplearning.ai!!!>

Stop words

and

is

are

at

has

for

a

Punctuation

,

.

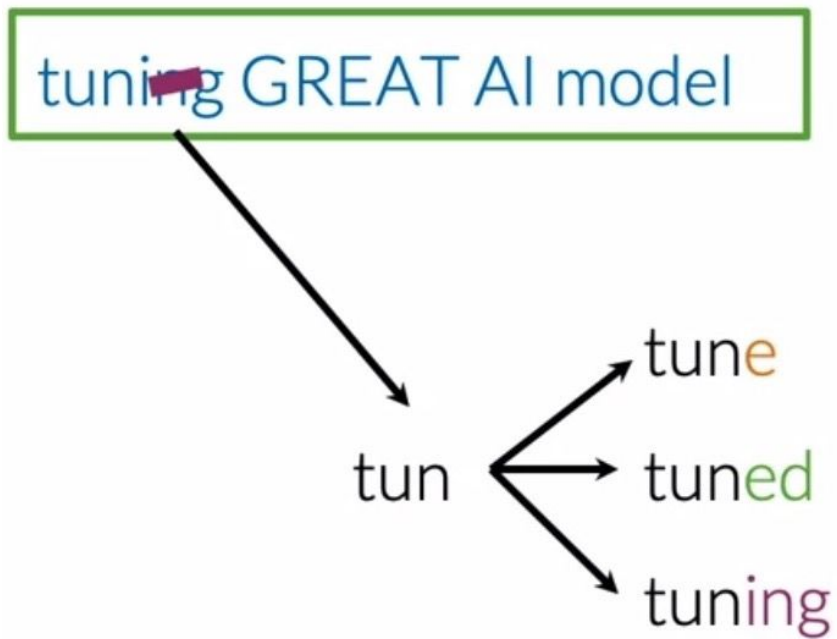
:

!

“

‘

Stemming



Lower-casing



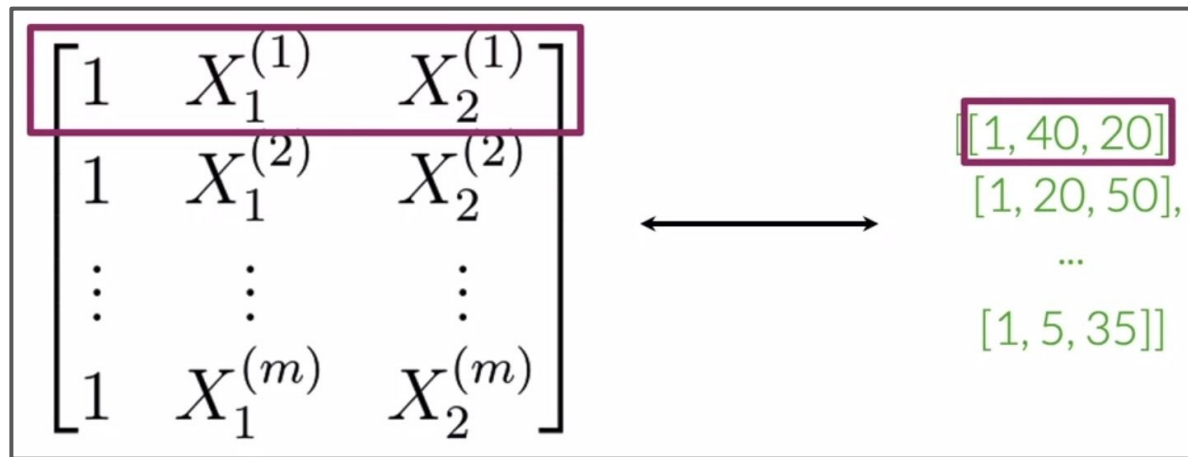
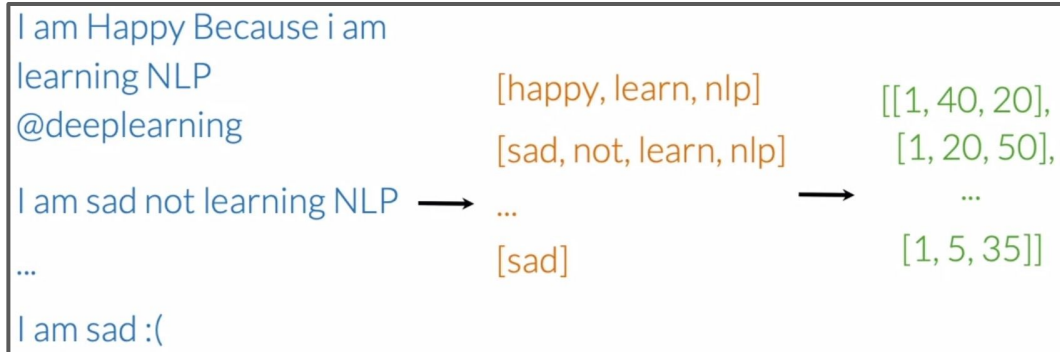
A diagram illustrating the lower-casing process. On the left, three words are listed vertically: 'GREAT', 'Great', and 'great'. A vertical line is positioned to the right of these words. A horizontal arrow points from the middle of this vertical line to the word 'great' on the right, indicating the transformation of the input words into their lower-case form.

GREAT
Great
great

great

Preprocessed tweet:
[tun, great, ai, model]

Generating the Features Matrix



Classification Using Logistic Regression

In linear regression, we'd predict:

$$h_{\theta}(x) = \theta^T x$$

But this can output any value from $-\infty$ to $+\infty$
and probabilities must be between 0 and 1.

So, to “squash” this linear combination into the range (0,1), we use the sigmoid function.

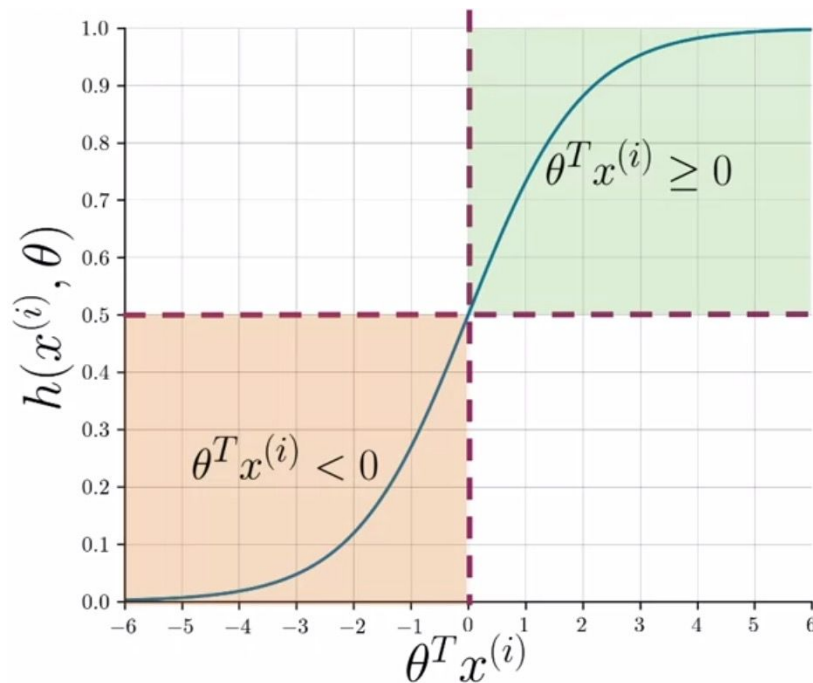
Sigmoid Function (a.k.a Logistic Function)

$$h(x^{(i)}; \theta) = \sigma(x^{(i)}; \theta) = \frac{1}{1 + e^{-\theta^T x^{(i)}}}$$

- Output range $\rightarrow (0, 1)$

That's why logistic regression is named after the logistic (sigmoid) function.

It converts the linear output into a probability.

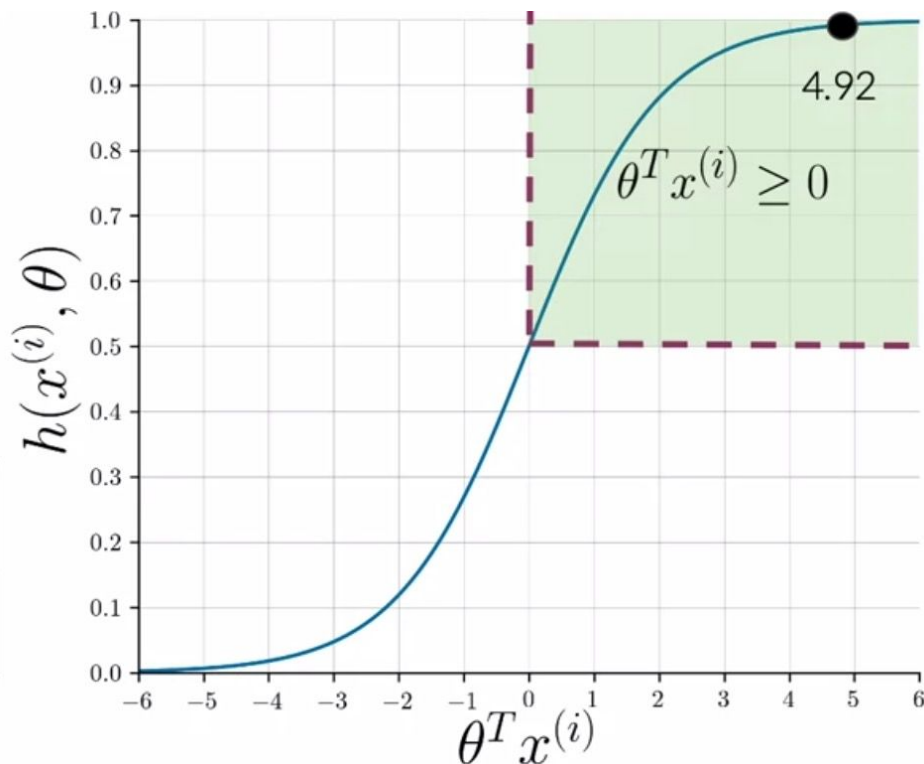


Example

@YMourri and
@AndrewYNg are tuning a
GREAT AI model

[tun, ai, great, model]

$$x^{(i)} = \begin{bmatrix} 1 \\ 3476 \\ 245 \end{bmatrix} \quad \theta = \begin{bmatrix} 0.00003 \\ 0.00150 \\ -0.00120 \end{bmatrix}$$



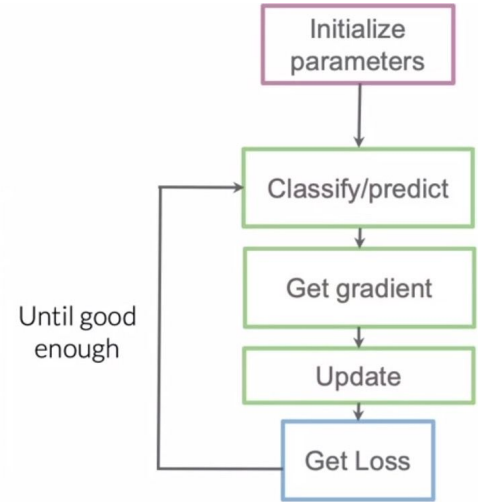
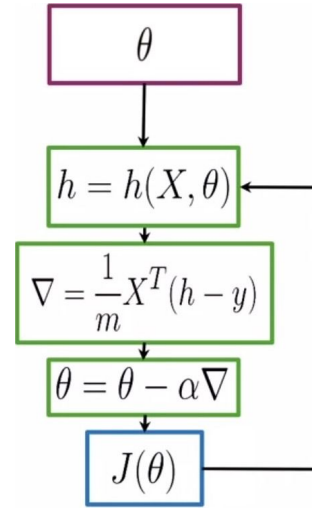
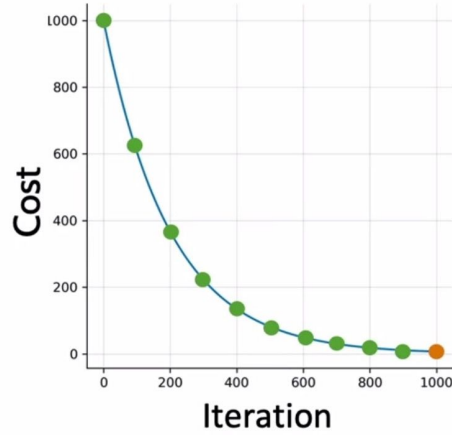
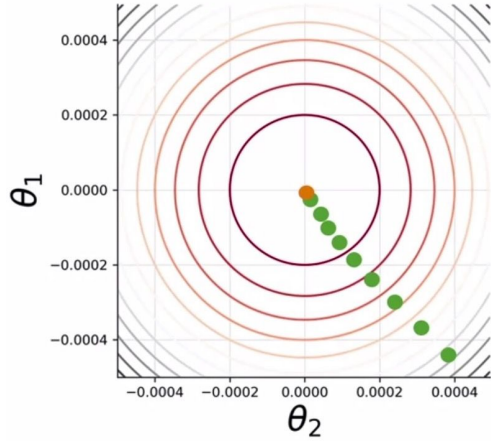
Computing Gradients

$$\frac{\partial J(\theta)}{\partial \theta_j} = \nabla_{\theta_j} J(\theta) = \frac{1}{m} \sum_{i=1}^m (y^{\hat{(i)}} - y^{(i)}) x_j$$

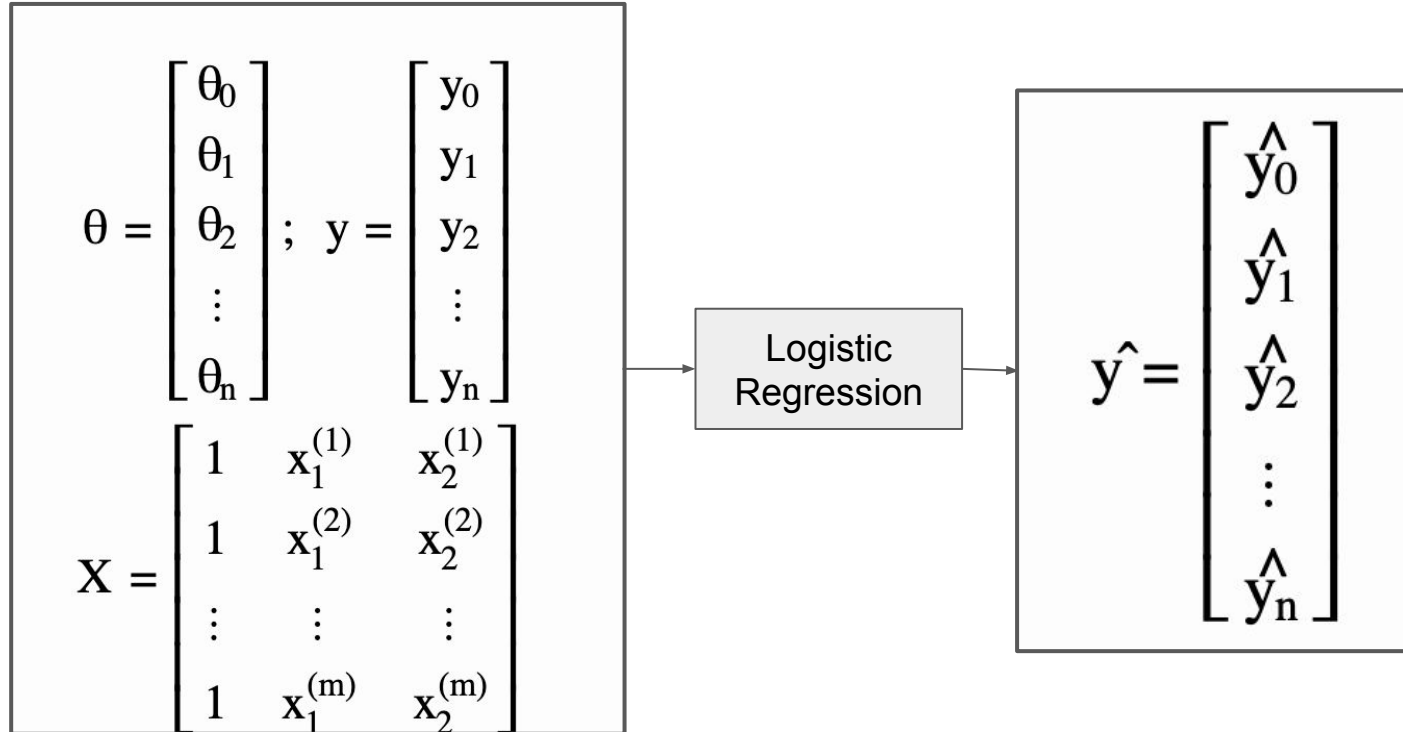
Update Rule

$$\theta_j := \theta_j - \alpha \times \nabla_{\theta_j} J(\theta)$$

Training Logistic Regression



Vectorization



Compute the cost function

$$J(\theta) = \frac{-1}{m} \times (y^T \cdot \log(\hat{y}) + (1 - y)^T \cdot \log(1 - \hat{y}))$$

$$\theta := \theta - \alpha \times \nabla_{\theta} J(\theta)$$

Calculate gradients and update rules

$$\frac{\partial J(\theta)}{\partial \theta} = \nabla_{\theta} J(\theta) = \frac{1}{m} \times (X^T \cdot (\hat{y} - y))$$

$$\theta := \theta - \alpha \times \left(\frac{1}{m} \times (X^T \cdot (\hat{y} - y)) \right)$$

Testing Logistic Regression

- To evaluate the generalization ability of the classifier and thus, test the model, generate predictions on unseen data (data that the model has not come across during training) and check how well it fares.

X_{val} Y_{val} θ

$h(X_{val}, \theta)$

$pred = h(X_{val}, \theta) \geq 0.5$

$$\begin{bmatrix} 0.3 \\ 0.8 \\ 0.5 \\ \vdots \\ h_m \end{bmatrix} \geq 0.5 = \begin{bmatrix} \underline{0.3 \geq 0.5} \\ 0.8 \geq 0.5 \\ 0.5 \geq 0.5 \\ \vdots \\ pred_m \geq 0.5 \end{bmatrix} = \begin{bmatrix} \underline{0} \\ 1 \\ 1 \\ \vdots \\ pred_m \end{bmatrix}$$

Compute the model's accuracy on the validation set

$$Y_{val} = \begin{bmatrix} 0 \\ 1 \\ \underline{1} \\ 0 \\ 1 \end{bmatrix} \quad pred = \begin{bmatrix} 0 \\ 1 \\ \underline{0} \\ 0 \\ 1 \end{bmatrix} \quad (Y_{val} == pred) = \begin{bmatrix} 1 \\ 1 \\ \underline{0} \\ 1 \\ 1 \end{bmatrix}$$

$$\text{accuracy} = \frac{4}{5} = 0.8$$

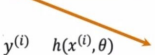
Cost Function Intuition

- The cost function used for logistic regression is the average of the log loss across all training examples. Formally:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log h(x^{(i)}, \theta) + (1 - y^{(i)}) \log(1 - h(x^{(i)}, \theta))]$$

- There is only one term in the cost function that is relevant when your label is 0 which is $-\log(\hat{y})$,
- and another that is relevant when the label is 1 which is $-\log(1-\hat{y})$.
- Intuitively, when your prediction is close to the label value, the loss is small, and conversely when your label and prediction disagree, the overall cost goes up.

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log h(x^{(i)}, \theta) + (1 - y^{(i)}) \log(1 - h(x^{(i)}, \theta))]$$



$y^{(i)}$	$h(x^{(i)}, \theta)$	
0	any	0
1	0.99	~ 0
1	~ 0	-inf

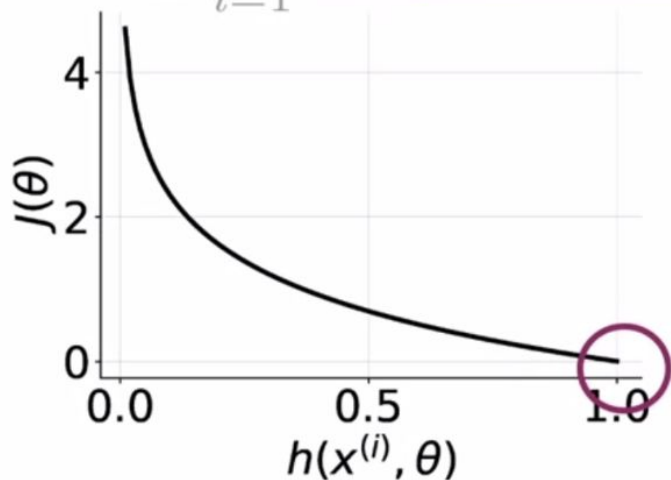
$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log h(x^{(i)}, \theta) + (1 - y^{(i)}) \log(1 - h(x^{(i)}, \theta))]$$



$y^{(i)}$	$h(x^{(i)}, \theta)$	
1	any	0
0	0.01	~ 0
0	~ 1	-Inf

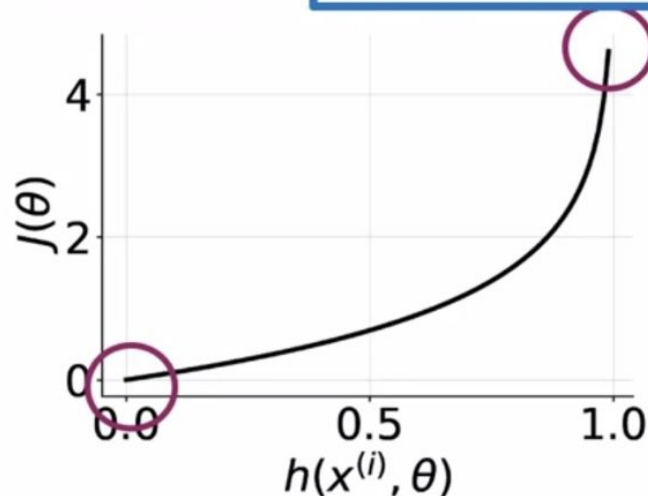
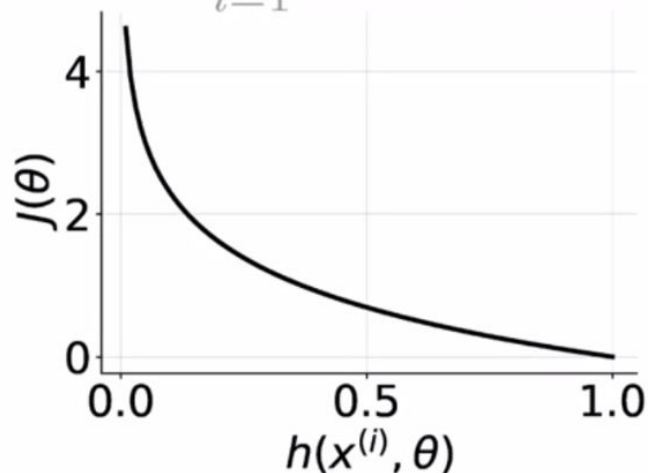
$$\text{if } y = 1 \implies J(\theta) = L(\hat{y}, 1) = -\log(\hat{y}) = \begin{cases} 0 & \text{if } \hat{y} = 1 \\ \infty & \text{if } \hat{y} = 0 \end{cases}$$

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log h(x^{(i)}, \theta) + (1 - y^{(i)}) \log(1 - h(x^{(i)}, \theta))]$$



$$\text{if } y = 0 \Rightarrow J(\theta) = L(\hat{y}, 0) = -\log(1 - \hat{y}) = \begin{cases} 0 & \text{if } \hat{y} = 0 \\ \infty & \text{if } \hat{y} = 1 \end{cases}$$

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log h(x^{(i)}, \theta) + (1 - y^{(i)}) \log(1 - h(x^{(i)}, \theta))]$$



Code Example

[https://colab.research.google.com/drive/1rDPjj-PYucnmMqvEV-jmMlwIroVHO-6_ ?
usp=sharing](https://colab.research.google.com/drive/1rDPjj-PYucnmMqvEV-jmMlwIroVHO-6_?usp=sharing)

Task

https://colab.research.google.com/drive/14UAB-DY1t_HG68kVek1wjeU847BywipM?usp=sharing